

Automated Security Verification System

To make the TEAVS–ADCRS security testing repeatable, an automated security tester was created in the backend repository.

Tester location:

security-verification/teavs_adcrs_security_tester.py

The tester was designed to verify the main security controls protecting:

POST /api/ingestion/core

The automated checks covered:

- API Gateway availability
- missing JWT handling
- invalid JWT handling
- RBAC denial for the analyst role
- successful access for the ingestion_service role
- incomplete payload rejection
- incorrect field type rejection
- malformed JSON rejection

The tester used the same controlled test accounts that were created during the manual RBAC verification.

Initial Automated Test Result

The first automated run identified two failed validation checks:

- incomplete payloads were accepted with HTTP 202
- payloads containing incorrect field types were accepted with HTTP 202

These failures confirmed the input validation issue that had already been observed during manual testing.

The validation middleware was then updated to match the actual CoreModelIntegrationPayload interface used by PHOENIX.

The payload interface requires:

- url
- text
- timestamp
- hazard_type
- hazard_severity
- hazard_timestamp
- hazard_location
- hazard_status
- alert_level
- source

The hazard_severity field is required to be numeric, while the remaining fields are expected to be strings.

Final Automated Test Result

After the validation control was implemented and the API Gateway was rebuilt, the automated tester was executed again.

The final result was:

Automated Test	Result
API Gateway health check	PASS
Missing JWT rejected	PASS
Invalid JWT rejected	PASS
Analyst blocked by RBAC	PASS
Ingestion service allowed	PASS
Incomplete payload rejected	PASS
Incorrect field types rejected	PASS
Malformed JSON rejected	PASS

Total tests: 8

Passed: 8

Failed: 0

This confirmed that the authentication, RBAC and payload validation controls were functioning as expected for the tested scenarios.

Engineering Value

The automated tester provides a repeatable way to verify the TEAVS–ADCRS integration security controls after future backend changes.

Instead of relying only on manual curl commands, the same set of security checks can now be rerun using:

```
python3 security-verification/teavs_adcrs_security_tester.py
```

This makes it easier to detect regressions in authentication, authorisation and request validation.

Evidence

EVID-AUTO-01 – Automated TEAVS–ADCRS Security Verification

Insert the screenshot showing the final automated tester output with:

Passed: 8

Failed: 0

Total: 8

```

(base) abdulrehman@Abduls-Air backend % python3 - <<'PY'
from pathlib import Path

p = Path("security-verification/teavs_adcrs_security_tester.py")
text = p.read_text()

old = '''    status, _ = request(
        "POST",
        "/api/ingestion/core",
        {},
        ingestion_token,
    )
    test("Ingestion service allowed", 202, status)'''

new = '''    valid_payload = {
        "url": "https://example.com/teavs-alert",
        "text": "TEAVS detected a potential phishing hazard",
        "timestamp": "2026-09-27T07:55:00+10:00",
        "hazard_type": "phishing",
        "hazard_severity": 4,
        "hazard_timestamp": "2026-09-27T07:54:00+10:00",
        "hazard_location": "VIC",
        "hazard_status": "active",
        "alert_level": "critical",
        "source": "TEAVS"
    }

    status, _ = request(
        "POST",
        "/api/ingestion/core",
        valid_payload,
        ingestion_token,
    )
    test("Ingestion service allowed", 202, status)'''

if old not in text:
    raise SystemExit("Tester block not found")

p.write_text(text.replace(old, new))
print("Tester updated with valid CoreModelIntegrationPayload")
PY
Tester updated with valid CoreModelIntegrationPayload
(base) abdulrehman@Abduls-Air backend % python3 security-verification/teavs_adcrs_security_tester.py

PHOENIX TEAVS-ADCRS SECURITY TESTER

[PASS] API Gateway health: expected 200, got 200
[PASS] Missing JWT rejected: expected 401, got 401
[PASS] Invalid JWT rejected: expected 401, got 401
[PASS] Analyst blocked by RBAC: expected 403, got 403
[PASS] Ingestion service allowed: expected 202, got 202
[PASS] Incomplete payload rejected: expected 400, got 400
[PASS] Wrong field types rejected: expected 400, got 400
[PASS] Malformed JSON rejected: expected 400, got 400

=====
TEST SUMMARY
=====
Passed: 8
Failed: 0
Total: 8
(base) abdulrehman@Abduls-Air backend %

```